Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3331076.3331112

RESEARCH-ARTICLE

Mining complex temporal dependencies from heterogeneous sensor data streams

AMINE EL OUASSOULI, University of Lyon, Lyon, Auvergne-Rhone-Alpes, France

LIONEL ROBINAULT, Foxstream, Vaulx-en-Velin, France

VASILE-MARIAN SCUTURICI, University of Lyon, Lyon, Auvergne-Rhone-Alpes, France

Open Access Support provided by:

University of Lyon

Foxstream

PDF Download
3331076.3331112.pdf
06 March 2026
Total Citations: 1
Total Downloads: 84

Published: 10 June 2019

Citation in BibTeX format

IDEAS 2019: 23rd International Database Engineering & Applications Symposium
June 10 - 12, 2019
Athens, Greece

IDEAS '19: Proceedings of the 23rd International Database Applications & Engineering Symposium (June 2019)
https://doi.org/10.1145/3331076.3331112
ISBN: 9781450362498

Mining Complex Temporal Dependencies from Heterogeneous Sensor Data Streams

Amine El Ouassouli
Univ Lyon, INSA Lyon, Foxstream
LIRIS (UMR 5205 CNRS)
Villeurbanne, France
amine.el-ouassouli@insa-lyon.fr

Lionel Robinault
Foxstream
LIRIS (UMR 5205 CNRS)
Vaulx-En-Velin, France
l.robinault@foxstream.fr

Vasile-Marian Scuturici
Univ Lyon, INSA Lyon
LIRIS (UMR 5205 CNRS)
Villeurbanne, France
marian.scuturici@insa-lyon.fr

ABSTRACT

In addition to sensor heterogeneity, monitoring applications must handle different temporal data models (e.g time series, event sequences). In this paper, we address the problem of discovering directly actionable high level knowledge from such data. We model temporal information through interval-based streams describing environment states. We propose an approach to discover efficiently Complex Temporal Dependencies (CTD) between state streams, called CTD-Miner. A CTD is modeled similarly to a conjunctive normal form and describes temporal relations (time delays) between states. CTD-Miner is robust to temporal variability of data and uses a statistical independence test to determine the most appropriate time lags between states. This test is also used to perform pruning on sub-dependencies checking. Finally, we validate our approach via synthetic data and a case study in a real-world smart environment using outdoor cameras and real-time video processing.

CCS CONCEPTS

• **Information systems** → **Data streams**; **Temporal data**; **Sensor networks**; **Data analytics**; **Data stream mining**.

KEYWORDS

Data streams, temporal data, mining complex events

ACM Reference Format:
Amine El Ouassouli, Lionel Robinault, and Vasile-Marian Scuturici. 2019. Mining Complex Temporal Dependencies from Heterogeneous Sensor Data Streams. In *23rd International Database Engineering & Applications Symposium (IDEAS'19), June 10–12, 2019, Athens, Greece.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3331076.3331112

1 INTRODUCTION

Temporal knowledge discovery is an important task for a growing number of application domains where large volumes of time-stamped data can be generated. Smart environments are a typical example of such contexts. They refer to places or objects equipped with a sensor system monitoring one or more physical measures through data streams. These make it possible to obtain temporal description of the environment's characteristics evolution that is induced by temporal phenomena occurring within it. In this context, a knowledge discovery task consists in extracting non-trivial, expressive and concise patterns describing typical hidden temporal phenomena.

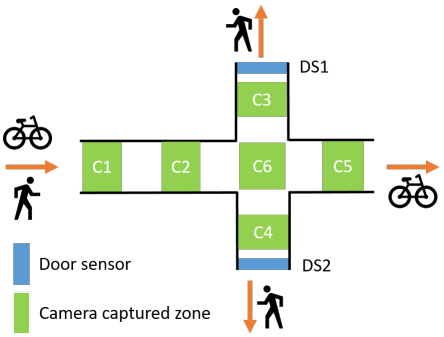


Figure 1: A smart environment

We describe in Fig. 1 several one-way corridors used by actors of two types: pedestrians and cyclists. This environment is equipped with a sensor system composed of door sensors and video cameras monitoring parts of the corridor. For data produced by video cameras, advances in image and video processing make it possible to obtain useful insights: motion detection, counts, object recognition. In this example, C1, C3, C4 and C5 provides motion detection, C2 provides object recognition and C6 counts moving objects. Each of these sensors provides a data streams to a monitoring application. Examples are shown in Fig.2. Our objective is to extracts temporal knowledge from such configurations.

Temporal descriptions of sensed environment are richer if different types of features are used. In the example described higher,

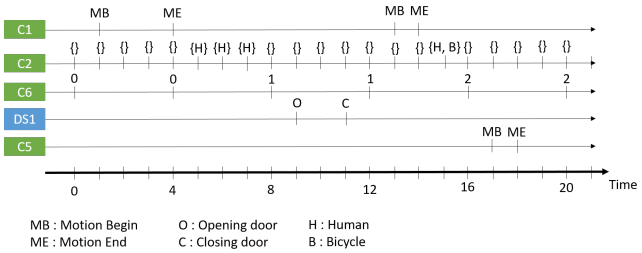


Figure 2: Some raw data streams gathered from the sensor system depicted in Fig.1

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
IDEAS'19, June 10–12, 2019, Athens, Greece
© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6249-8/19/06…$15.00
https://doi.org/10.1145/3331076.3331112

the analysis of motion information would permit to obtain 3 main trajectories *"C1 then C3"*, *"C1 then C4"* and *"C1 then C5"*. In this example, C3 and C4 can only be reached by pedestrians and C5 by cyclists. A more valuable insight is given by the heterogeneous relation *"C1 then <Bicycle> then C5"*. The usage of heterogeneous descriptions for sensed environments offers the opportunity to discover temporal knowledge reporting on complex relations between different features of the environment. This poses a challenging problem to temporal pattern mining approaches:

How to obtain complex, non-trivial and directly actionable temporal knowledge starting from heterogeneous sensor raw data?

Each sensor may use the most appropriate time model (time points or intervals) and data model (e.g numeric time series, symbolic events or item sets) w.r.t its physical measure. A simple way to address heterogeneity is to transform raw sensor data sequences to a unified general model using a Temporal Abstraction (TA) operation. In [9] authors defined TA as "*the segmentation and/or aggregation of series of raw (...) data into a symbolic time-interval series representation, often at a higher level of abstraction (...), suitable for human inspection or for data mining*". In addition to solving heterogeneity problems, TA permits to build high level *pattern vocabulary* that it more suitable for human perception and interpretation [5]. In this work, we use an interval-based representation built on states referring to data configurations of interest for the application domain. States are defined via predicates on one or more sensors producing data. A state stream contains parts of time (intervals) where the state's predicate is valid. This way, data provided by a sensor system is transformed to a set of unified interval-based state streams.

Time intervals allow to take into account straightforwardly duration in a discrete time contrary to point based events. Therefore, using a point-based approach to process interval data induces a loss of information and expressiveness. As discussed in [1], 13 relations can exist between two intervals. Existing interval-based qualitative pattern models, based on all or a subdivision of Allen' logic, may suffer from various expressiveness issues as ambiguity (a pattern may lead to different temporal relations) or completeness (capability of expressing all possible relations) [5]. In some extent, quantitative patterns maintaining temporal information permit to solve these problems as temporal relations are explicitly characterized permitting to infer Allen' logic. Moreover, time lags can also be a discriminant factor. In the example described higher, the trajectory *"C1 then C2"* can be performed by pedestrians and cyclists. These two types of actors performs the same qualitative trajectory but with different temporal information: cyclists *"C1 then C2 after d units of time"* are faster than pedestrians *"C1 then C2 after d' units of time"*, where $d < d'$. This can be often useful, for example, to perform forecasting: *"if C1 then C2 after d"*, one can predict that the following trajectory step is C5 (rule corresponding to cyclists).

Very few existing approaches tackled directly quantitative interval patterns in streams or can easily be adapted to this task ([11], [4]). In [11], authors introduced a novel form of temporal relations based on the assessment of intervals intersection. The intersection of a pair of state streams contains time portions where both states are active. The length of this intersection is used to assess statistically the significance of the temporal correlation. In our work, this model is extended to handle more complex relations involving multiple states. We want to express relations including disjunctions and conjunctions involving multiple data streams, like *"IF A then (B or C) after a duration d"*.

In this work, we firstly introduce the Complex Temporal Dependency model (CTD), a quantitative pattern model as an extension of pairwise dependencies proposed in [11]. It is automatically assessed using a χ^2 test of independence on interval intersection length. Next, we propose CTD-Miner, an algorithm devised to discover efficiently CTDs. It includes a novel time lag discovery method, sub-dependencies pruning techniques and dependencies merging. Finally, we conducted experiments using both simulated and real life motion sensor data to validate our approach.

2 RELATED WORK

Sequential and temporal pattern mining is a well studied research area designed to discover regularities among temporally ordered data. Contributions in this field can be categorized following three main criteria: data format (transaction databases or streams/single sequences), time models (time points or intervals) and temporal description (qualitative or quantitative).

Most of existing approaches consider input data to be in a transactional format: each transaction is a temporally ordered sequence associated with an ID. A typical example would be medical data: a sequence describes medical history of a particular patient identified by an ID. Transactional format makes clear separations between activities (e.g between patients medical records) and can be called "subject-centered". With this data format, temporal pattern mining task consists on finding temporal regularities between transactions. On the other hand, sensor streams (or single sequences) are a continuous flow of data where no boundary exists between activities and describe the evolution of physical measures. For example, a motion sensor reports on motion rather than describing motion of objects separately: a same piece of data can be generated by one or various actors activity. Hence, sensor data can be called "measure-centered". In both cases, existing contributions use a user-given minimum support referring to the part of data where a relation stands. The support assessment can be based on number of transaction for transactions, time windows [7] or number of items [13].

Most of the contributions dealing with streaming data or single sequences focus on time points with qualitative patterns reporting on *before/after/co-occurring* relations [7]. Quantitative patterns do not permit a time delay discrimination which is rather useful for many application domains. While some contributions integrate temporal constraints (time windows [7], gap constraints [2]), several approaches tackled quantifying time delays ([6] [13] [14]). Another category of contributions deals with data represented as intervals. As discussed in [1] this time model permits to express more complex relations (13 relation types in Allen' logic). This form of patterns may suffer from expressiveness limitations as ambiguity (a pattern may lead to different relations) or completeness (not allowing to express all possible relations). We refer to [5] for a detailed analysis of expressiveness. Besides, qualitative patterns are sensitive to temporal variability as slight variation in intervals endpoints may lead to different qualitative relations.

Quantitative patterns permit to deal with these problems as they include temporal information permitting to infer Allen' logic.

Among the existing contributions [10] [3] [12] only two recent approaches are designed or can be easily adapted to interval streams [4] [11]. In [4], the authors propose PIVOTMiner that uses a geometric approach consisting on the projection of each interval $[t_{begin}, t_{end})$ into a bi-dimensional plane $(begin, end)$ where the temporal relation between two intervals is considered as a vector. This allows to perform a DBSCAN clustering after an origin transformation stage: to mine relations of type $A \rightarrow B$, all vectors with A as source and B as target are moved such as all sources coincides with the space origin. Cluster centroides provides the time lag information. While designed for sequence databases, this approach can be easily adapted to data streams as it is not endpoint sensitive (as in [12], [3]). Finally, in [11] the authors proposed a novel form of temporal relations made possible by the usage of intervals. It is based on the intersection of sets of validity intervals corresponding to predicates describing the environment states. The authors propose an algorithm, TEDDY, devised to discover dependencies of type *"$A \rightarrow B$ after (α, β) units of time"* assessed via a confidence measure.

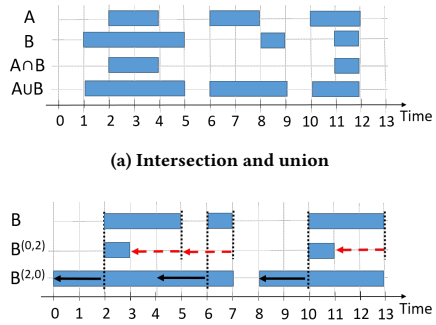
3 STATE STREAMS

3.1 From raw sensor data to state streams

We define a data stream D as a sequence of time stamped data produced by a source $d \in \Lambda$, with Λ the set of data sources composing a sensor system. Formally, a data sequence S_d produced by d is defined with $S_d = \{\langle(t, v)\rangle \mid t \in \mathcal{T}, v \in \mathcal{V}_d\}$. $\mathcal{T}$ is an infinite set of discrete time stamps and t can be either a time point or an interval $t = [t_b, t_b + 1)$. $\mathcal{V}$ is the set of possible values given by d. We assume that a data source cannot produce more than one value at a time. In the example depicted in Fig 1, the set of data sources is $\Lambda = \{C1, C2, ...DS1, DS2\}$. Possible values for the data source $C1$ is a set of event labels $\mathcal{V}_{C1} = \{MB, ME\}$, for $C2$ is all subsets of possible objects $\mathcal{V}_{C2} = \{O_i \mid O_i \subseteq \{H, B\}\}$, and for $C6$ is the set of positive integers $\mathcal{V}_{C6} = \mathbb{N}$. Examples of state streams are provided in Fig 2. Notice that data sources can be heterogeneous at different levels: sample rates, data models (e.g time series, labeled events, item sets), time models (time points or intervals). Our objective is to process data streams given by Λ in order to obtain high level temporal knowledge.

Our approach to solve problems introduced by heterogeneity is to use Temporal Abstraction (TA) devised to transform raw heterogeneous data into a unified interval based model using high level abstraction providing a "pattern vocabulary" for temporal knowledge. This technique was often applied for medical records (e.g [9]) or with sensor time series (e.g [8]). More precisely, the main idea is to use expert knowledge or automatic techniques (e.g time series discretization) to define a batch of high level states that can be seen as "environment features" of interest. A state is a particular environment configuration referring to an non-trivial situation. Formally speaking, a state a is defined as a predicate: $a : \mathcal{T}, \Lambda \rightarrow \mathbb{B}$. The result of a is a boolean value stating that the state is whether active or inactive at $[t, t + 1) \in \mathcal{T}$ given a set of data sequences in Λ. Some examples of states:

- **motionInC1**$(t, C1)$ ::= $last(t, C1).v = MB$
- **increasingOccupency**$(t, C6)$ ::= $last(t, C6).v - last(t - 1, C6).v > 0$



(a) Intersection and union

(b) (α, β)-transformation

Figure 3: Examples of intersection and union and (α, β)-transformation

- **congestion**$(t, C1, C6)$ ::= $(last(t, C1).v = MB) \wedge (last(t, C6).v - last(t - 1, C6).v = 0)$

Predicates can be simple conditions on a single raw data sequence values as *motionInC1* that reports on simple motion activity (the last event produced by $C1$ at the timestamp t is MB=Motion Begin). States can also report on data trends as *increasingOccupency*, or can also integrate data from various sensors as for *congestion*.

A state stream is a temporally ordered sequence containing all time intervals, called active intervals, where its corresponding state predicate is verified. A state stream A corresponding to state a is formally defined as $A = (a(t, \lambda), \langle[t_{b_i}, t_{e_i})\rangle)$ such that $\forall t_{b_i}, t_{e_i} \in \mathcal{T}, t_{b_i} < t_{e_i} < t_{b_{i+1}}$ and $\forall t \in [t_{b_i}, t_{e_i}) \mid a(t, \lambda) = True$ with $\lambda \subseteq \Lambda$. The size of a state stream A noted $\#A$ corresponds to the number of its intervals. The length of a state stream is the sum of its active intervals duration:

$$len(A) = \sum_{[t_b, t_e) \in A} (t_e - t_b)$$

For example, the state stream corresponding to $motionInC1$ from Fig. 2 is $motionInC1 =< [1, 4), [13, 14) >$, with $size(motionInC1) = 2$ and $len(motionInC1) = 4$.

3.2 Operations on state streams

We define several operations on state streams: intersection, union and temporal transformation.

The intersection of two state streams A and B, noted $A \cap B$ is a state stream containing intervals where both A and B are active (Fig.3). Formally, $A \cap B = (a \wedge b, \langle[t_{b_i}, t_{e_i})\rangle$ such that $\forall t \in [t_{b_i}, t_{e_i}), \exists [t_{b_j}, t_{e_j}) \in A, [t_{b_k}, t_{e_k}) \in B$ such that $t \in [t_{b_j}, t_{e_j})$ and $t \in [t_{b_k}, t_{e_k})$. This operation is computed in $\Theta(Max(\#A, \#B))$.

The union of two state streams A and B, noted $A \cup B$, produces a new state stream containing the intervals where A or B are active (Fig. 3). Formally, $A \cup B = (a \wedge b, \langle[t_{b_i}, t_{e_i})\rangle$ such that $\forall t \in [t_{b_i}, t_{e_i}), \exists [t_{b_j}, t_{e_j}) \in A, [t_{b_k}, t_{e_k}) \in B$ such that $t \in [t_{b_j}, t_{e_j})$ or $t \in [t_{b_k}, t_{e_k})$. Similarly to intersection, the union of two state streams has a time complexity of $\Theta(Max(\#A, \#B))$.

A state stream B can also be temporally shifted via an (α, β)-transformation. This operation results on a state stream $B^{(\alpha,\beta)} = \langle[t_{b_i} - \alpha, t_{e_i} - \beta) \mid [t_{b_i}, t_{e_i}) \in B\rangle$. Hereafter, α is called expansion and

β reduction. We describe in Fig. 3 two examples: a $(0, 2)$-reduction $B^{(0,2)}$ and a $(2, 0)$-expansion $B^{(2,0)}$. This temporal transformation is done in $\Theta(\#B)$.

4 COMPLEX TEMPORAL DEPENDENCIES

This section introduces the Complex Temporal Dependencies (CTD) model that aims to describe temporal correlations between multiple state streams on the basis of their intersection length.

4.1 Background

A pairwise temporal dependency between state streams A and B, noted $A \rightarrow B$, notifies that *A occur simultaneously with B*. We call hereafter A the premise and B the conclusion. $A \rightarrow B$ is assessed with the intersection length of A and B via the following confidence measure:

$$conf(A \rightarrow B) = \frac{len(A \cap B)}{len(A)}$$

Notice that confidence is maximal (=1) if all active intervals of A are included in active intervals of B: A occurs *always* with B. For example, in Fig. 3 $conf(A \rightarrow B) = \frac{3}{6}$.

In order to find relations of interest, and to avoid the utilization of an user-given threshold, this confidence measure is statistically assessed via a Pearson χ^2 test of independence. The independence hypothesis states that A and B are statistically independent within a duration $\mathcal{T}$. It relies on the following assumption: if active length of B is uniformly distributed in $\mathcal{T}$, there is no significant correlation between A and B. The given validity threshold on confidence is noted $th(len(A), len(B))$[1] and obtained for a significance level of 0.05 and 1 degree of freedom [11].

Simultaneity do not permit to express dependencies when it happens that B is time-delayed regarding A. The conclusion stream can be temporally shifted with an (α, β)-transformation to obtain a relation of type $A \rightarrow B^{(\alpha,\beta)}$. Therefore, the associated confidence value is given by:

$$conf(A \rightarrow B^{(\alpha,\beta)}) = \frac{len(A \cap B^{(\alpha,\beta)})}{len(A)}$$

A dependency $A \rightarrow B^{(\alpha,\beta)}$ means that: B starts at most α time units after A and B finishes at least β time units after A. For simplicity purposes, we refer hereafter to this relation with: if A is active then $B^{(\alpha,\beta)}$ is active.

4.2 Conjunctive and disjunctive relations

As previously described, pairwise state dependencies are composed of a premise, a conclusion and an inference operator "$\rightarrow$" that is associated with a confidence measure and describes simultaneity and succession relations. Our goal is to express dependencies with multiple states.

In Fig.4 pairwise dependencies are $A \rightarrow B^{(2,3)}, conf = \frac{3}{5}$ and $B \rightarrow C^{(3,2)}, conf = \frac{6}{9}$. In this example, C always follows a succession "A then B": a dependency *A followed by B then C* should have a

[1] $th(lp, lc) = \dfrac{lp * lc + \sqrt{\frac{3.84}{T} lp * lc(T_{obs} - lc)(T_{obs} - lp)}}{T_{obs} * lp}$

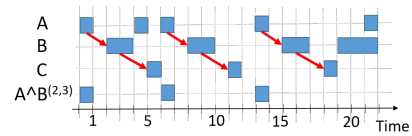


Figure 4: Example of 3 correlated streams

confidence of 1. This example permits to emphasize that dependencies between multiple states must be able to express conditional relations like: *state C is correlated with state B* ***if*** *B is preceded by state A after a duration d*. This is made possible by the introduction of the conjunctive operator $\wedge$ that expresses simultaneity or succession, but contrary to the inference it is not associated with a confidence measure. A conjunction of two state streams A and B, noted $A \wedge B$, corresponds to the intersection $A \cap B$. This conjunctive stream can be seen as the stream representation of $A \rightarrow B$ and be used to extend this dependency. In the higher example, we can extract the following conjunctive dependencies:

$$A \wedge B^{(2,3)} \rightarrow C^{(5,5)}, conf = 1 \quad (1)$$

$$A \rightarrow B^{(2,3)} \wedge C^{(5,5)}, conf = \frac{3}{6} \quad (2)$$

These two dependencies are to be interpreted differently: (1) If state A and $B^{(2,3)}$ are active **then** $C^{(5,5)}$ is active with a confidence of 1; (2) If A is active **then** $B^{(2,3)}$ and $C^{(5,5)}$ are simultaneously active with a confidence of $\frac{3}{6}$.

A more expressive form of dependencies is made possible by the introduction of disjunctive relations, noted $\vee$. Let us consider temporal configuration of states described in Fig 5. Using conjunctive relations allows us to obtain the following dependencies: $A \wedge B \rightarrow C \wedge D$, $A \wedge B \rightarrow E$, $A \wedge B \rightarrow F \wedge G \wedge H$, $A \wedge B \rightarrow F \wedge G \wedge I$. This states configuration can be concisely described with a unique dependency using disjunctive relation stating:

```
IF   A AND B are active
THEN (C AND D are active) OR
     (E is active) OR
     (F and G and (H OR I) are active)
```

The corresponding dependency is noted as follows :

$$\begin{aligned} A \wedge B^{(\alpha_b,\beta_b)} \rightarrow & (C^{(\alpha_c,\beta_c)} \wedge D^{(\alpha_d,\beta_d)}) \vee \\ & E^{(\alpha_e,\beta_e)} \vee \\ & (F^{(\alpha_f,\beta_f)} \wedge G^{(\alpha_g,\beta_g)} \wedge (H^{(\alpha_h,\beta_h)} \vee I^{(\alpha_i,\beta_i)})) \end{aligned}$$

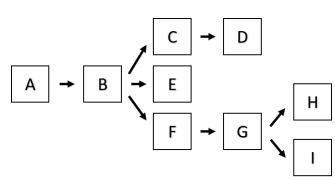


Figure 5: A temporal configuration of states. A **is followed by** B, B **is followed by** C **or** E **or** F **etc...**

Notice that the premise and the conclusion of this dependency are both state streams. It suffices to compute, for each dependency part, conjunctions (stream intersection), then disjunctions (stream union) to obtain a single state stream representative. This allows us to compute the confidence value following the same principle as for pairwise dependencies.

Complex Temporal Dependencies (CTD) are defined as dependencies including conjunctive and disjunctive temporal relations:

Definition 4.1. **Complex Temporal Dependency**.
Let $\mathcal{S} = \{A_1, A_2, ..., A_n\}$ be a set of state streams. A Complex Temporal Dependency (CTD) over $\mathcal{S}$ is defined with:

$$D_0 \wedge D_1 \wedge ... \wedge D_k \rightarrow D_{k+1} \wedge ... \wedge D_{k+m}$$

with D_i a state stream resulting of disjunctive and conjunctive operations using a subset $\mathcal{S}_i \subset \mathcal{S}$ such that $\forall i, j\ \mathcal{S}_i \cap \mathcal{S}_j = \emptyset$. Temporal transformations are defined with respect to a stream from D_i having a $(0, 0)$-temporal transformation.

4.3 Sub-dependencies and correspondence relationship

Conjunctive relations make it possible to obtain dependencies between multiple states reporting on "large" significant temporal correlations. As a consequence, a same temporal phenomenon involving n states can be described entirely with a single dependency or partially with multiple smaller dependencies, called sub-dependencies. We describe in Fig.6 three state streams A, B and C producing the following dependencies:

$$A \rightarrow B^{(2,2)}, conf = 0.5 \quad (3)$$

$$B \rightarrow C^{(2,2)}, conf = 1 \quad (4)$$

$$A \rightarrow C^{(4,4)}, conf = 1 \quad (5)$$

$$A \wedge B^{(2,2)} \rightarrow C^{(4,4)}, conf = 1 \quad (6)$$

In this example, dependencies (3) and (4) are sub-dependencies of (6): intervals involved in (3) and (4) are included in (6). On the other hand, dependency (5) is not a sub-dependency of (6): half of intervals of A and C do not intervene in (6). The ability to detect sub-dependencies is a key feature in a dependency discovery process that can be compared to closure-checking for support-based algorithms. In addition to reducing the amount of redundant information (pattern flooding), the sub-dependency property can be useful to define pruning criteria to accelerate a discovery process. Sub-dependency checking can be done via the study of conjunction intersection via the following *correspondence* relationship:

Definition 4.2. **Correspondence relationship**
Let A and B be two state streams. A and B are corresponding if

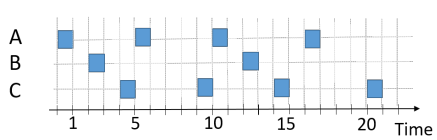


Figure 6: Dependencies $A \rightarrow B^{(2,2)}$ and $B \rightarrow C^{(2,2)}$ are included in $A \wedge B^{(2,2)} \rightarrow C^{(4,4)}$. $A \rightarrow C^{(4,4)}$ is not.

$$conf(A \rightarrow B) \geq 1 - \epsilon \text{ and } conf(B \rightarrow A) \geq 1 - \epsilon$$

where ϵ is a relaxation parameter such that $\epsilon \ll 1$.

Correspondence between A and B permits to assess whether two state streams are exclusively co-occurring: A occur "almost" always with B and inversely. The correspondence checking is referred to as a boolean function $Corr : A, B, \epsilon \rightarrow \mathbb{B}$. The ϵ parameter allows an error rate in the correspondence checking to tackle noisy or temporally variable relations. This parameter can be defined with respect to the statistical threshold on confidence measure. It corresponds to the minimum intersection length that can be considered as statistically significant. The loss of a smaller amount w.r.t the maximal confidence value (=1) can be considered as non-significant.

The sub-dependency checking between two dependencies D_1 and D_2 comes to evaluate if the representative conjunctions of both dependencies are correspondent. In the higher example, $A \wedge B^{(2,2)}$ and $A \wedge B^{(2,2)} \wedge C^{(4,4)}$ are correspondent with maximal confidence values:

$$conf(A \wedge B^{(2,2)} \rightarrow A \wedge B^{(2,2)} \wedge C^{(4,4)}) = 1$$

$$conf(A \wedge B^{(2,2)} \wedge C^{(4,4)} \rightarrow A \wedge C^{(2,2)}) = 1$$

On the other hand, $A \wedge C^{(4,4)}$ and $A \wedge B^{(2,2)} \wedge C^{(4,4)}$ are not correspondent:

$$conf(A \wedge C^{(4,4)} \rightarrow A \wedge B^{(2,2)} \wedge C^{(4,4)}) = 0.5$$

$$conf(A \wedge B^{(2,2)} \wedge C^{(4,4)} \rightarrow A \wedge C^{(4,4)}) = 1$$

In these examples, the temporal reference stream is A (all temporal transformation are expressed w.r.t to intervals of A) for both dependencies. Otherwise, a temporal transformation and an intersection are applied. For example, a $(2, 2)$ transformation is applied to $B \wedge C^{(2,2)}$ as B is shifted with a $(2, 2)$-transformation in $A \wedge B^{(2,2)} \rightarrow C^{(4,4)}$. The sub-dependency checking is done w.r.t the intersection of this transformed stream and A (in this case: $A \wedge B^{(2,2)} \wedge C^{(2,2)}$ and $A \cap (B \wedge C^{(2,2)})^{(2,2)}$).

Definition 4.3. **Sub-dependency** Let $\mathcal{S} = \{A_1, A_2, ..., A_n\}$ be a set of state streams, R_1 a CTD with streams in $\mathcal{S}_1 \subset \mathcal{S}$ and R_2 a CTD with streams in $\mathcal{S}_2 \subset \mathcal{S}$. Then A_{R_2} A_{R_1} are respectively the temporal reference of R_1 and R_2. A_{R_2} have a (α, β)-transformation in R_1. R_2 is a sub-dependency of R_1 if $\mathcal{S}_2 \subseteq \mathcal{S}_1$ and :

$$Corr(R_1, A_{R_1} \cap {R_2}^{(\alpha,\beta)}, \epsilon) = True$$

5 PROBLEM DEFINITION

The proposed dependency model defines a search space that can be characterized in spatial and temporal dimensions. The temporal dimension contains all possible temporal transformations (α, β). Its total size for a discrete observation duration $\mathcal{T}$ is $|\mathcal{T}|^2$ as $\alpha \in \mathcal{T}$ and $\beta \in \mathcal{T}$. The spatial dimension includes all combination between state streams and dependencies operators (inference, conjunctive and disjunctive relations) in addition to the temporal dimension for all streams. This defines a very large search space even for a small set of state streams. If only conjunctive relations are taken

into account, the search space size is given by:

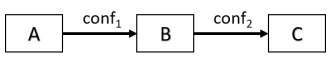
$$\sum_{k=2}^{n} \frac{n!}{(n-k)!} * (k-1) * |\mathcal{T}|^{2k}$$

with n the number of input streams. The number of conjunctive relations without temporal transformations is given by the number of arrangements of 2 to n elements from a set of n state streams. The factor $(k-1)$ stands for the possible inference operator position within a dependency. For example, with A, B and C, one can obtain $A \wedge B \rightarrow C$ or $A \rightarrow B \wedge C$ which are two different dependencies. Moreover, for each state in a conjunctive dependency, there are $|\mathcal{T}|^2$ possible streams to be taken into account. This search space size shows that performing a naive exploration is not affordable. Besides, this number is a lower bound of the total search space size as it does not take into account all possible disjunctive relations.

In order to perform a feasible CTD discovery, we assume that is not useful to look for dependencies with large time gaps between states (similar to a window size constraint in [7]). Therefore, we limit the allowed time lag to an interval $\Delta = [min, max]$ such that $\alpha, \beta \in \Delta$ defining a quadratic search space of size $|\Delta|^2$ for each stream. In addition to this temporal constraint, we limit our search space to dependencies with the following form:

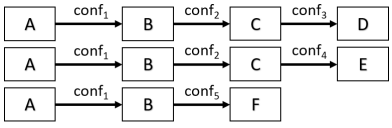
$$S_0 \wedge S_1 \wedge S_2 ... \rightarrow S_k$$

We consider that such dependencies provide sufficient insights to describe temporal phenomena. For example, for a state succession A, B then C, obtaining $A \rightarrow B, conf_1$ and $A \wedge B \rightarrow C, conf_2$ permit to characterize this temporal phenomenon at each of its steps is associated with a corresponding confidence measure. This kind of characterization can be represented as a directed acyclic graph:

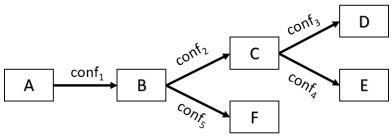


In this example, node A corresponds to state stream A, B to conjunction $A \wedge B$ and C to $A \wedge B \wedge C$. Edge corresponds to confidence measure: $conf_1 = conf(A \rightarrow B)$ and $conf_2 = conf(A \wedge B \rightarrow C)$. This representation permits to obtain a condensed representation of a given temporal phenomenon including temporal transformations.

Moreover, we assume that statistically valid disjunctive relations can be obtained only from statistically valid conjunctions. In other words, if A is temporally correlated with $B \vee C$, A is correlated with B and A is correlated with C. For example, in Fig. 7.a one can notice that the conjunctive relation $A \rightarrow B$ is common to the 3 temporal relations. The idea is to perform of a factorization of dependencies common conjunctive relations using the correspondence relationship. The resulting disjunctive dependencies can be represented as a tree. With the former example, the corresponding tree representation is depicted in Fig. 7b. Notice that all information provided by the three conjunctive relations are kept by this disjunctive tree representation. Therefore, this paper tackles the following problem: Given a set $\mathcal{S}$ of state streams and a temporal constraint on time lags $\Delta = [min, max]$, extract all conjunctive Complex Temporal Dependencies of the form $S_0 \wedge S_1 \wedge S_2 ... \rightarrow S_k$ and build disjunctive tree representations of corresponding temporal phenomena.



(a) Conjunctive temporal relations



(b) Corresponding disjunctive form

Figure 7: 3 conjunctive temporal relations and corresponding disjunctive form

6 CTD-MINER

CTD-Miner is based on an incremental approach consisting on building dependencies with $i+1$ conjunction from previously computed dependencies with i conjunctions (Algorithm 1).

6.1 CTD incremental construction

The incremental construction of conjunctive relations starts with considering all streams in $\mathcal{S}$ as premise candidates (line 2). In the main loop (line 3 to 21), each pair of premise candidate p and stream $s \in \mathcal{S}$ are tested via a time lag discovery algorithm (line 13). Notice that a state label is allowed to appear only once in a dependency via the non-cyclic condition in line 12 (the conclusion state label must not appear in the premise). Results given by significant time lag discovery are used to extend p in *ExtendDependency* (line 16) that creates a new premise candidate $p \wedge r$. In order to be able to reconstruct the graph representation, the extension of dependency stores previous confidence values and temporal transformations in addition to conjunctive streams. If no results are given by the time lag discovery algorithm, p is added to the results set: p is no longer extendable. New premises at a given iteration are considered as premise candidates for the next iteration (line 21). The loop ends when no premise candidates are left. The non-cyclic condition (line 12) guarantees the termination of the main loop.

The disjunctive relations constructions constitute the final step of CTD-Miner. The results obtained after the main loop are composed of conjunctive relations. We recall that conjunctions resulting at each extension step are stored. Disjunctive dependencies are built using the correspondence relationship if two dependencies D_1 and D_2 share their k first conjunctive relation labels and the resulting conjunction are correspondent their are merged. Finally, the result set $\mathcal{R}$ is returned (line 24).

6.2 Correspondence relationship based pruning

The above steps constitute the baseline version of CTD-Miner that implements the incremental construction of conjunctive relations. The resulting dependencies may contain sub-dependencies that are considered as redundant information. For example, let us consider a temporal phenomenon, called A, constituted of n temporally consecutive states $A_1 \rightarrow A_2 ... \rightarrow A_n$. The results set will contain dependencies describing all or a subdivision of this phenomenon,

Algorithm 1: CTD-Miner

Input: $\mathcal{S}$: a set of state streams
$\mathcal{T}$: observation duration
$\Delta = [min, max]$: temporal constraint on time lags
Output: $\mathcal{R}$: set of CTDs

1 $\mathcal{R} \leftarrow \emptyset$
2 premises $\leftarrow \mathcal{S}$
3 **while** $|premises| > 0$ **do**
4 $newPremises \leftarrow \emptyset$
5 **for** $p \in premises$ **do**
6 $extended \leftarrow False$
7 $process \leftarrow True$
8 **if** $|p| > 1$ **then**
9 $process \leftarrow$ PrePruning($p, newPremises \cup \mathcal{R}$)
10 **if** $process$ **then**
11 **for** $s \in \mathcal{S}$ **do**
12 **if** $s.label \notin p.labels$ **then**
13 results $\leftarrow$ TimeLagDiscovery(p, s, Δ, $\mathcal{T}$)
14 **for** $r \in results$ **do**
15 $extended \leftarrow True$
16 $ext \leftarrow$ ExtendDependency(p, r)
17 **Add** ext **to** $newPremises$
18 **if** $|p| > 1$ ***and not*** $extended$ **then**
19 **Add** p **to** $\mathcal{R}$
20 $newPremises \leftarrow$ MergeDependencies($newPremises$)
21 $premises \leftarrow newPremises$
22 $\mathcal{R} \leftarrow$ PostProcessing($\mathcal{R}$)
23 $\mathcal{R} \leftarrow$ BuildDisjunctions($\mathcal{R}$)
24 **return** $\mathcal{R}$

$A_{n-1} \rightarrow A_n$, $A_{n-2} \wedge A_{n-1} \rightarrow A_n$ ... $A_1 \wedge A_2 ... \wedge A_{n-1} \rightarrow A_n$. Besides, for each of the previous dependencies, the conjunction relations order can differ while maintaining temporal information: $A_1 \wedge A_3^{(\alpha_3,\beta_3)} \rightarrow A_2^{(\alpha_2,\beta_2)}$ and $A_1 \wedge A_2^{(\alpha_2,\beta_2)} \rightarrow A_3^{(\alpha_3,\beta_3)}$ corresponds to the same temporal succession of states. Therefore, the total number of resulting conjunctive dependencies describing this n-states temporal phenomenon is given by:

$$|\mathcal{R}|_A = \sum_{k=2}^{n}(k-1)! = \sum_{k=1}^{n-1} k!$$

In order to keep the number of solutions moderate, CTD-Miner benefits from a sub-dependency checking at two levels. First, as a pre-pruning step (line 9) via the PrePruning function that performs a sub-dependency checking of a premise p w.r.t dependencies in $newPremises$ and $\mathcal{R}$. If p is a sub-dependency of another, it is pruned: *PrePruning* returns false and p is not processed nor added to the result set. Second, to ensure that no sub-dependencies are left in the final results, a post processing step (line 22) is performed.

Another way to benefit from the correspondence relationship is that of merging dependencies (line 20). This operation consists of extending dependencies with other dependencies rather than a single stream. For example, $A \rightarrow B^{(\alpha_b,\beta_b)}$ and $B \rightarrow C$ can be merged if $A \wedge B^{(\alpha_b,\beta_b)}$ and $A \cap (B \wedge C)^{(\alpha_b,\beta_b)}$ are correspondent.

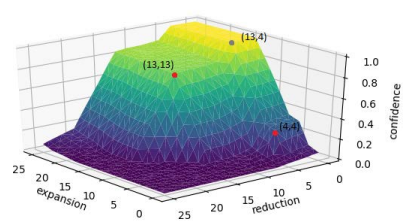
6.3 Time lag discovery

The discovery of temporally quantitative correlation between two state streams is a core operation in CTD-Miner. This operation is executed at a high frequency during the discovery process. As a consequence, its time complexity influences significantly the performance of CTD-Miner.

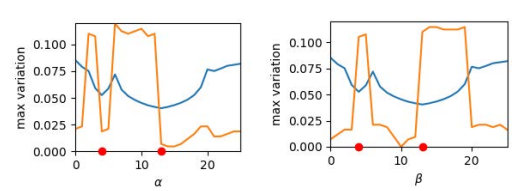
In order to discover such correlations, we designed the ITLD (Interval Time Lag Discovery) heuristic algorithm that we describe briefly in this section. It performs a linear significant time lag discovery w.r.t Δ. The main idea of this algorithm is to detect statistical significant confidence gains and losses. More precisely, given a pair of state stream A and B, for each i $\alpha \in \Delta$, ITLD computes the following confidence variations:

$$gain = conf(A \rightarrow B^{(\alpha,\alpha)}) - conf(A \rightarrow B^{(\alpha+1,\alpha)}$$
$$loss = conf(A \rightarrow B^{(\alpha,\alpha)}) - conf(A \rightarrow B^{(\alpha,\alpha+1)})$$

The confidence gains and losses are calculated w.r.t a non deformed conclusion streams i.e with (α, α)-transformations. This permits to guarantee that intervals of the conclusion B cannot be merged or canceled due to an important deformation induced by temporal transformations, which constitute a loss of transition information. These confidence variations are qualified as elementary as they are obtained by adding a unit to whether the expansion (for gains) or the reductions (for losses). As a consequence, the conclusion gained/lost length corresponds to number of intervals of B. The statistical assessment of these elementary confidence variations is done with the validity threshold $th(len(A \cap B^{(\alpha,\alpha)}, \#B)$ is it aims to evaluate whether the confidence variation is statistically independent w.r.t the gained/lost conclusion length. We describe in Fig. 8 the entire



(a) Temporal search space



(b) Resulting confidence variations (orange) and statistical thresholds (blue). Red points refer to significant expansion and reduction values

Figure 8: Temporal search space (a) and resulting confidence variations for a pair of dependencies with two temporal relations $(13, 13)$ **and** $(4, 4)$**.** $\Delta = [0, 25]$

search space (a) and the corresponding confidence gains, losses and thresholds.

7 EXPERIMENTS

This section presents the experimental results that illustrate the efficiency of CTD-Miner. We experimented our approach using synthetic data provided by a motion simulation tool that makes it possible to obtain data corresponding to different scenarios. We also study the behaviour of CTD-Miner with a real world motion data set generated from a sensor system composed of outdoor video cameras and using real time video processing. Algorithms were implemented in Python[2] and tested on a Core i7 2.1Ghz with 8GB memory running Windows 10.

7.1 Simulated Data

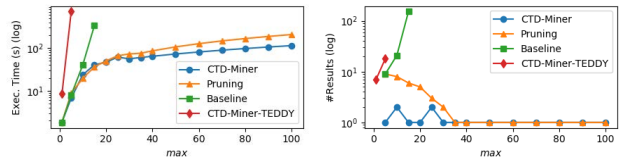
We developed a motion simulation tool[2] in order to obtain data sets using custom scenarios (trajectories, activity density, speed of moving objects, temporal variability). This controlled testbed generates data sets with a known ground truth permitting to evaluate the accuracy of discovered temporal dependencies in a multitude of configurations.

Table 1 describes datasets obtained from the simulation tool. We defined a linear trajectory with 10 equidistant sensors and ran 11 simulations varying the number of occurrences for the same duration $\mathcal{T} = 10000$ and object speed (cf Table 1). Each dataset contains 10 streams and the typical time lag between successive state is $\approx (4, 4)$. The number of intervals increases with the number of event occurrences for sparse data (100 to 5000 occurrences) and decreases for high density event occurrences due to intervals overlap. The streams active length always increases when the event occurrences increases and have a density ($\frac{len(stream)}{T_{obs}}$) ranging from 1.7% to 78.5%. It is to notice that the expected result from these datasets is the trajectory description including 10 states with the corresponding (α, β)-transformations (represented as a 10 nodes graph).

Hereafter, we evaluate our approach with respect to size of Δ, density of streams intervals and scalability. We also examined the impact of sub-dependencies pruning, dependencies merging and the time lag discovery algorithm on CTD-Miner's efficiency. To this end, we consider the following CTD-Miner configurations: **Baseline**: consists of the simple incremental construction of CTD as described in section 6.1; **Pruning**: line 20 is removed from Algorithm 1 and only the sub-dependencies pruning is performed ; **CTD-Miner**: CTD-Miner as described in Algorithm 1. ITLD was used for the three previous algorithm. The fourth CTD-Miner configuration is **CTD-Miner-TEDDY** with TEDDY [11] instead of ITLD. For the following tests, we limited the execution of the CTD discovery algorithms to 20 minutes and used the statistical threshold for the correspondence relationship.

7.1.1 Influence of the size of Δ. Fig.9 reports the running time and number of conjunctive relations (before disjunctions construction) for the 1000 occurrences dataset when $\Delta = [0, max]$ varies.

[2]https://github.com/AElOuassouli/Quantitaive-Interval-Stream-Mining



(a) Running time **(b) Number of results for different values of** max

Figure 9: Running time and number of results of CTD-Miner, Pruning, Baseline, CTD-Miner-TEDDY w.r.t to max

CTD-Miner-TEDDY and Baseline reached the 20 minutes running time limit for respectively $max = 10$ and $max = 20$. In both cases, the number of premise candidates, and by extension the number of time lag discovery executions, increases significantly. This can be noticed with the number of the given conjunctive relations. TEDDY outputs a non-negligible amount of false positives w.r.t the simulation scenario. For example, with $\Delta = [0, 1]$ TEDDY obtained 7 dependencies when none is expected. Candidate sub-dependencies increases significantly for the Baseline as the number of given conjunctive dependencies shows.

CTD-Miner and Pruning were capable of completing the discovery process for all values of $max \in [0, 100]$ with an advantage for CTD-Miner. Results for Pruning corresponds to dependencies with time lags in a given $\Delta = [0, max]$ while CTD-Miner returns a unique dependency corresponding to the expected 10 conjunctions dependency for $max \geq 5$ (except for 10 and 25 where Δ did not permit to fully capture the intersection for a temporal relation with $\alpha = max + 1$ and $\beta = max + 1$, causing the non-verification of the correspondence relationship). It is to notice that Pruning obtains the same result for $max \geq 35$. Thus, thanks to the sub-dependency pruning and the dependency merging CTD-Miner scales well with respect to Δ.

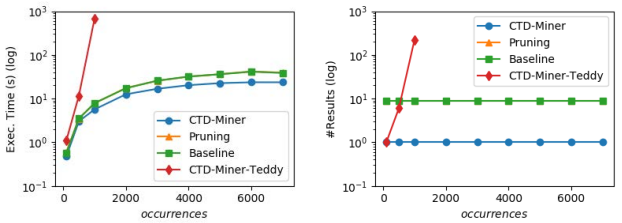
7.1.2 Influence of density of streams. As described in Table 1, activity density (number of occurrence) comes generally with an increase of length and intervals number. Therefore, a complex temporal dependency discovery algorithm must be capable of scaling with respect to intervals numbers and be able to detect precisely temporal phenomena in both sparse and dense state streams.

Fig.10 reports the empirical study of CTD-Miner with respect to streams density. The constant number of conjunctive results given by CTD-Miner with the ITLD (CTD-Miner, Pruning and Baseline) for each value of max shows that ITLD is robust to density of streams and allows CTD-Miner to scale well with respect to intervals number for these datasets. This is not the case for CTD-Miner-Teddy due to its time lag discovery algorithm (that 20 minutes limit was reached for 2000 occurrence with $max = 5$).

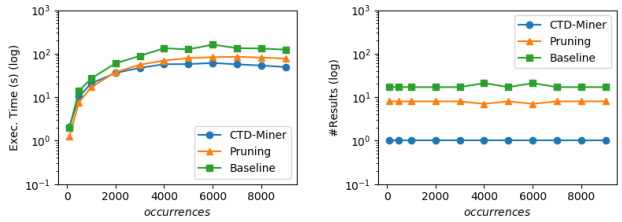
7.1.3 Scalability. One important aspect of CTD discovery is that of the ability of processing large numbers of streams. We generated a data set of 90 streams following the same principle as data sets described in Table 1 with $\mathcal{T} = 10000$ time units, 1000 occurrences and a time lag of $\approx (6, 6)$ between successive states. Fig.11 shows that CTD-Miner was able to process 90 streams (containing 63990

Table 1: 11 simulated data sets with increasing occurrences number (*Occ*). *#Int*: average number of intervals per stream, *Den*: average density % of T_{obs}

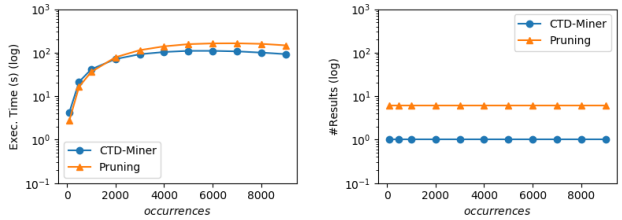
Occ	#Int	Den
100	95	1.7
500	452	8.7
1000	798	16.4
2000	1275	30.3
3000	1509	41.5
4000	1606	50.5
5000	1616	58.6
6000	1570	64.7
7000	1493	69.9
8000	1386	74.6
9000	1263	78.5



(a) $max = 5$



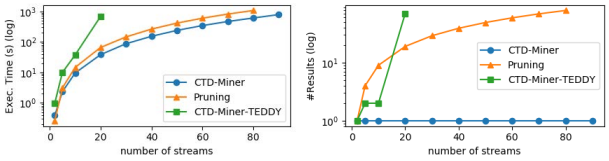
(b) $max = 9$



(c) $max = 15$

Figure 10: Running time and Number of conjunctive results with respect to occurrence number with different values of *max*

intervals) within the 20 minutes time limit. CTD-Miner returned a unique dependency describing the entire trajectory, with all available streams, for all input sizes thanks to its merging procedure contrary to Pruning which is limited by the temporal constraint $\Delta = [0, 9]$ and returns an increasing number of pairwise dependencies. Notice that for this particular Δ, the performances of Pruning are equivalent to Baseline as the sub-dependency pruning is not performed. CTD-Miner-TEDDY reaches the 20 minutes for 30 streams and returns the trajectory description in addition to a large number of dependencies that are considered as false positives w.r.t to the simulation scenario.



(a) Running Time (b) Number of conjunctive relations

Figure 11: Running time and number of conjunctive relations w.r.t to number of streams. $\Delta = [0, 9]$

7.2 Real life motion data

We describe in Fig. 12 and Fig. 13 a sensor system composed of 4 outdoor cameras situated in an office area. These cameras are capturing motion using real time video processing. Starting from images taken by these cameras, we defined 10 "virtual" motion sensors (displayed with red polygons) corresponding to physical regions and labelled as 1-1, 1-2, ..., 4-3. Each virtual motion sensor produces a sequence of time point events of two types: *"B: Motion Begin"* and *"E: Motion End"*. We defined for each area X an

Figure 12: Four outdoor camera views. Red contours describe motion analysis areas.

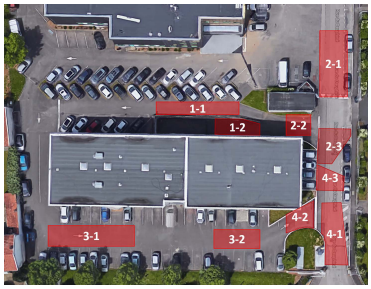


Figure 13: Position of motion analysis areas (aerial view)

Table 2: Dataset corresponding to the experiment described in Fig.13

Name	#Int	Len
1-1	1403	4303
1-2	851	3257

Name	#Int	Len
2-1	8640	47881
2-2	2099	4947
2-3	8548	26847

Name	#Int	Len
3-1	3909	14644
3-2	3699	13423

Name	#Int	Len
4-1	9686	30257
4-2	4578	13397
4-3	9825	21273

environment state *Motion in area X* noted **M-X** and defined with **M-X**(t,X) ::= $last(t, X).v == B$. The 10 resulting state streams are described in Table 2. This data set describes motion activity in the office area during 18 working days between 6 am and 8 pm. First observations showed that it contains a significant amount of noise (e.g detection of shadows, sudden luminosity changes) and several omissions were observed (e.g when a car passes through an analysis zone with a great speed). Moreover, the resulting streams are extremely sparse and low statistical thresholds impacts the correspondence relationship. As a consequence, we used $\epsilon = 0.2$ as relaxation parameter. The low statistical thresholds impacts to some extent the precision of ITLD: thresholds were in some cases lowers than significant confidence variations. In our experiments we used the statistical thresholds [1] with a significance level of 0.005 (critical value of $\chi^2_{0.005} = 7.88$ instead of $\chi^2_{0.05} = 3.84$) to obtain more precise results.

The execution of CTD-Miner with $\Delta = [0, 5]$ was completed in 300 seconds and provided 12 disjunctive relations. Figure 14 provides two examples that illustrate two behaviours: entering office area (a) and leaving office area (b). What is to notice is that the first dependency was built using the incremental construction of CTDs all time lags are included in Δ and the second using dependencies merge. Moreover none of the given dependencies provides false information (e.g relation between *2-1* and *3-1*). We emphasize the fact that the discovery process was completed in a negligible amount of time in comparison with the observation duration.

8 CONCLUSION AND FUTURE WORK

We proposed an approach to discover complex temporal dependencies (CTD) starting from interval based state streams. These streams are built through temporal abstractions on heterogeneous data. Each temporal abstraction (state) is defined using a predicate defining a state of interest for the application domain. We introduced the Complex Temporal Dependencies model that models temporal relations between state streams and their typical time delays using conjunctive and disjunctive relations. We also proposed CTD-Miner, an incremental algorithm that is devised to CTD discovery. To validate our model and the discovery process, we conducted several experiments on both simulated and real motion sensor data. From these results, we conclude that CTD-Miner speeds up the exploration process using its linear time delay discovery, its sub-dependency pruning and dependencies merge. The encouraging results given for the real data set show that it is possible to integrate video analysis methods in a data analysis process which opens perspectives for a wide range of application scenarios. For example, in a commercial context, our approach may permit to investigate temporal dependencies between user-given streams as commercial results or client satisfaction rate, sensory data streams as motion, people counting, and video processing-based streams as objects classification (adults, stroller).

In a future work, we intend to design an on-line version of CTD-Miner in order to obtain up-to-date models of temporal phenomena occurring within a sensed environment. This would permit to investigate the ability to detect behavior drifts (emergence of new dependencies) in addition to the forecasting capabilities of our intersection-based dependency model.

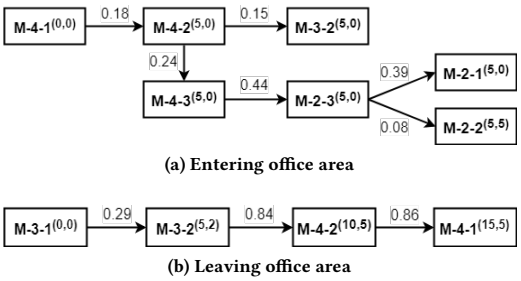


Figure 14: Example given by CTD-Miner on real world data generated by environment depicted in Figure 13

REFERENCES

[1] J. F. Allen and P. J. Hayes. A common-sense theory of time. In *IJCAI*, pages 528–531. Morgan Kaufmann, 1985.
[2] G. Casas-Garriga. Discovering unbounded episodes in sequential data. In *PKDD*, volume 2838 of *Lecture Notes in Computer Science*, pages 83–94. Springer, 2003.
[3] T. Guyet and R. Quiniou. Extracting temporal patterns from interval-based sequences. In *IJCAI*, pages 1306–1311. IJCAI/AAAI, 2011.
[4] M. Hassani, Y. Lu, J. Wischnewsky, and T. Seidl. A geometric approach for mining sequential patterns in interval-based data streams. In *FUZZ-IEEE*, pages 2128–2135. IEEE, 2016.
[5] F. Höppner and S. Peter. Temporal interval pattern languages to characterize time flow. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.*, 4(3):196–212, 2014.
[6] T. Li and S. Ma. Mining temporal patterns without predefined time windows. In *ICDM*, pages 451–454. IEEE Computer Society, 2004.
[7] H. Mannila, H. Toivonen, and A. I. Verkamo. Discovery of frequent episodes in event sequences. *Data Min. Knowl. Discov.*, 1(3):259–289, 1997.
[8] F. Mörchen. Algorithms for time series knowledge mining. In *KDD*, pages 668–673. ACM, 2006.
[9] R. Moskovitch and Y. Shahar. Fast time intervals mining using the transitivity of temporal relations. *Knowl. Inf. Syst.*, 42(1):21–48, 2015.
[10] F. Nakagaito, T. Ozaki, and T. Ohkawa. Discovery of quantitative sequential patterns from event sequences. In *ICDM Workshops*, pages 31–36. IEEE Computer Society, 2009.
[11] M. Plantevit, C. Robardet, and V. Scuturici. Graph dependency construction based on interval-event dependencies detection in data streams. *Intell. Data Anal.*, 20(2):223–256, 2016.
[12] G. Ruan, H. Zhang, and B. Plale. Parallel and quantitative sequential pattern mining for large-scale interval-based temporal data. In *BigData*, pages 32–39. IEEE Computer Society, 2014.
[13] L. Tang, T. Li, and L. Shwartz. Discovering lag intervals for temporal dependencies. In *KDD*, pages 633–641. ACM, 2012.
[14] W. Wang, C. Zeng, and T. Li. Discovering multiple time lags of temporal dependencies from fluctuating events. In *APWeb/WAIM (2)*, volume 10988 of *Lecture Notes in Computer Science*, pages 121–137. Springer, 2018.